Numerical Analysis

김시조
Professor of Mathematics
Mississippi State University

Prologue

Currently the lecture note is not fully grown up; other useful techniques and interesting examples would be soon incorporated.

Note. You may use any of my lecture notes as textbooks or references.
https://github.com/seejokim1/NA_with_python/tree/main

Contents

Chapter 1

파이썬의 이해

1.1 파이썬의 특징

Overview

Python is one of the most intuitive and flexible tools among modern programming languages, especially establishing itself as a powerful tool in engineering calculations such as numerical analysis.

Key Characteristics:

- Platform independence
- Concise syntax
- Dynamic typing
- Interpreter-based execution
- Extensive library ecosystem

Why Python for Numerical Analysis?

- Can perform identical numerical calculations across various environments with the same code
- Immediate result verification through code writing and execution
- Provides great advantages for learners and researchers

Key Libraries for Numerical Analysis:

- **NumPy:** Supports high-performance numerical operations including vector and matrix operations
- **pandas:** Helps easily manipulate and analyze complex datasets

- **Matplotlib:** Provides tools for intuitively understanding results through data visualization
- **SciPy:** Provides advanced computation functions needed for mathematical, scientific, and engineering problems

Conclusion: Python significantly enhances engineering problem-solving capabilities by providing extensive tools and libraries for easily handling complex numerical analysis problems.

From a Numerical Analysis Perspective:

- Tool for quickly and simply solving engineering problems
- Simplifies repetitive operations with simple syntax
- Users can focus on algorithm implementation
- Efficiently handles entire numerical problem-solving process

1.1.1 플랫폼 독립성

Definition

Python code can obtain identical results on various operating systems such as Windows, macOS, and Linux.

Example: Simple file reading code works identically on all operating systems

```
with open('example.txt', 'r') as file:
    print(file.read())
```

Click here GitHub Source Code for print

Advantages:

- Write once, run anywhere
- No platform-specific modifications needed
- Ideal for collaborative projects across different systems

1.1.2 인터프리터 언어

Definition

You can write Python code and immediately execute it to check results.

Example: Simple calculation in Python shell

```
print(2 + 3)
# Output: 5
```

Benefits:

- Rapid prototyping and testing
- Interactive development
- Immediate feedback for learning
- No compilation step required

1.1.3 부호와 줄바꿈

Indentation in Python

Python uses indentation to define blocks instead of braces {}.

Indentation Rules:

- Code within a block should be indented by 4 spaces or a tab
- Indentation size must be consistent within the same block
- Mixed indentation causes errors
- Use `pass` keyword for empty blocks

Comparison:

- C/C++/Java: Use braces {}
- MATLAB: Use if-else-end structure
- Python: Use indentation only

1.1.4 객체지향 프로그래밍 지원

Defining classes and creating objects:

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "makes a sound"

class Dog(Animal):
    def speak(self):
        return "barks"

pet = Dog("Buddy")
print(pet.name, pet.speak())
# Output: Buddy barks
```

1.1.5 동적 타입

Definition

Variables can be assigned and changed to various types of values without type declaration, making code writing more flexible and simple.

```
number = 10          # Initially assign an integer
print(number)        # Output: 10

number = "ten"       # Later assign a string
print(number)        # Output: ten
```

Advantages:

- More flexible and simpler code
- No explicit type declarations needed
- Quick prototyping

1.1.6 NumPy 라이브러리

Definition

NumPy is a library that enables high-performance numerical calculations in Python, optimized for large multidimensional arrays and matrix operations.

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a + b)  # Array addition
# Output: [5 7 9]
```

Key Features:

- High-performance array operations
- Broadcasting capabilities
- Extensive mathematical functions
- Foundation for scientific computing in Python

1.2 변수선언

Overview

This section explains variable declaration and usage in Python. Python automatically determines data types when values are assigned, reducing the burden of data type declaration and ensuring flexibility in program design.

Key Concepts:

- No explicit type declaration needed
- Data type determined automatically upon value assignment
- Type casting available through functions like `int()`, `float()`, `str()`
- Cannot use reserved keywords as variable names

Click here GitHub Source Code for variables

1.2.1 자료 형, 변수 선언과 변환

Python supports dynamic typing without type declaration:

```
x = 10                          # Integer
print(x)                        # Output: 10

y = 3.14                        # Float
print(y)                        # Output: 3.14

name = "Alice"                  # String
print(name)                     # Output: Alice

is_valid = True                 # Boolean
print(is_valid)                 # Output: True
```

Key Points:

- Even single characters are treated as strings (str)
- Easy conversion between numbers and strings
- Integers have no size limit

1.2.2 파이썬 예약어 (Keywords)

Definition

Reserved keywords are predefined words used to define the language's syntax and structure. These keywords cannot be used as variable or function names.

Python Keywords (33 total):

and	as	assert	break	class
continue	def	del	elif	else
except	exec	finally	for	from
global	if	import	in	is
lambda	nonlocal	not	or	pass
print	raise	return	try	while
with	yield			

Remember: Keywords are case-sensitive!

1.3 배열(Array)과 행렬(Matrix)

Overview

This section explains methods for handling arrays and matrices in Python and their applications.

Python Lists vs NumPy Arrays:

- **Lists:** Simple arrays, can store various data types, flexible but slower
- **NumPy Arrays:** High-performance arrays for large-scale data, optimized for mathematical operations

Key Topics:

- Basic array creation using lists
- Advanced array operations using NumPy
- Matrix operations
- Applications in data analysis and scientific computing

1.3.1 리스트(List)를 사용한 배열

List Format and Characteristics:

```
# Basic list format
list_name = [element1, element2, element3, ...]

# Examples
colors = ['red', 'green']           # String list
empty_list = []                      # Empty list
mixed_list = ['Hello', 1, 2]        # Mixed types
nested = [['Hello', 'World'], 1, 2] # Nested list
```

Characteristics:

- Can store various data types together
- Dynamic size adjustment
- Treated as row vector by default
- Not optimized for mathematical operations

```
a = [10, 20, 30, 40]
print(a[0])    # Output: 10 (first element)
print(a[-1])   # Output: 40 (last element)

# Operations
sum = a[0] + a[1]
print(sum)     # Output: 30

# Nested lists
a = [1, 2, ['a', 'b', ['Life', 'is']]]
print(a[2][2][0])   # Output: Life
```

Key Points:

- Indexing starts at 0
- Negative indices access from end
- Nested indexing for multidimensional access

1.3.2 리스트(List) 슬라이싱(Slicing)

Slicing Syntax

`list_name[start:end:step]`

- **start:** Starting index (inclusive)
- **end:** Ending index (exclusive)
- **step:** Index interval (default 1)

```
a = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(a[2:5])    # [2, 3, 4]
print(a[:4])     # [0, 1, 2, 3]
print(a[5:])     # [5, 6, 7, 8, 9]
print(a[::2])    # [0, 2, 4, 6, 8]
print(a[-3:])    # [7, 8, 9]
print(a[::-1])   # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

1.3.3 NumPy와 배열과의 관계

NumPy provides optimized array operations:

```
import numpy as np

# Create NumPy array
array_np = np.array([1, 2, 3, 4])
print(array_np)  # [1 2 3 4]

# 2D array
array_2d = np.array([[1, 2], [3, 4]])
print(array_2d)
# [[1 2]
#  [3 4]]

# Vectorized operation
result = array_np + 10
print(result)  # [11 12 13 14]
```

Advantages over lists:

- Memory efficient
- Faster operations
- Vectorized computations

1.3.4 행렬과 배열과의 관계

Matrix operations using NumPy:

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Matrix multiplication
C = A @ B
print(C)
# [[19 22]
#  [43 50]]

# Transpose
transpose = A.T
print(transpose)
# [[1 3]
#  [2 4]]
```

Key Operations:

- Matrix multiplication: `@` or `np.dot()`
- Transpose: `.T`
- Element-wise operations supported

1.4 튜플(tuple)

Definition

Tuples are similar to lists but are immutable (cannot be changed after creation), suitable for maintaining data integrity.

```
# Tuple creation
T = ((10, 20), (30, 40), (50, 60))
print('T=', T)
print('T[0][0]=', T[0][0])  # Output: 10

# Nested tuples
T4 = (1, 2, ('a', 'b', ('Life', 4)))
print(T4[2][2][0])  # Output: Life
```

Key Difference from Lists:

- Lists use square brackets [ ]
- Tuples use parentheses ( )
- Tuples are immutable

1.4.1 튜플이란?

1.4.2 튜플(tuple) 형식

1.4.3 튜플(tuple) 특징

Immutability:

```
T = ((10, 20), (30, 40))
# T[0][0] = 55  # This will raise TypeError
```

Single-element tuples need a comma:

```
T = (55,)       # Correct single-element tuple
print(T[0])     # Output: 55

T = (55)        # This is just an integer, not a tuple
# print(T[0])  # TypeError: 'int' object is not subscriptable
```

When to Use Tuples:

- Data that shouldn't be modified
- Function return multiple values
- Dictionary keys (tuples can be keys, lists cannot)

1.4.4 튜플(tuple) 예제

1.4.5 튜플(tuple) slicing : ":" 사용

1.4.6 튜플(tuple) Extended slicing : "::" 사용

1.5 사전(Dictionary)

1.5.1 딕셔너리(dictionary)

Definition

A dictionary is a data structure that stores data as key-value pairs, making data search and manipulation very fast and efficient.

Key Features:

- **Keys:** Must be immutable (strings, numbers, tuples)
- **Values:** Can be any data type
- **Speed:** Fast data search and modification
- **Mutable:** Can dynamically add, modify, or delete entries

Common Operations:

- Access: `dict[key]`
- Add/Modify: `dict[key] = value`
- Delete: `del dict[key]`
- Check existence: `key in dict`

Use Cases:

- Storing structured data
- Configuration settings
- Counting occurrences
- Caching results

1.5.2 딕셔너리(dictionary) 예제

실습 코드: https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_05_dictionary.py

```
# Dictionary creation
student = {
    "name": "Alice",
    "age": 25,
    "major": "Engineering"
}

# Value access
print(student["name"])  # Output: Alice

# Value modification
student["age"] = 26

# Adding new key-value pair
student["grade"] = "A"

# Deleting a key
del student["major"]

print(student)  # {'name': 'Alice', 'age': 26, 'grade': 'A'}
```

1.6 수학 연산자 및 NumPy 비교

1.6.1 파이썬의 기본 수학 연산자와 NumPy 연산 비교

Overview

This section explains the differences between Python's basic mathematical operators and the advanced numerical computation library NumPy.

Operation	Python	NumPy
Addition	`a + b`	`np.add(a, b)`
Multiplication	`a * b`	`np.multiply(a, b)`
Division	`a / b`	`np.divide(a, b)`
Power	`a ** b`	`np.power(a, b)`
Square root	-	`np.sqrt(a)`
Matrix mult.	-	`np.dot(a, b)` or `a @ b`

1.6.2 Python 과 NumPy의 기본 연산

실습 코드: https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_06_operators_numpy.py

1.6.3 Python 과 NumPy의 배열 연산

Python lists vs NumPy arrays for large-scale operations:

```
import numpy as np
import time

# Python list operation
a = list(range(1_000_000))
b = list(range(1_000_000))
start = time.time()
c = [a[i] + b[i] for i in range(len(a))]
print("List operation:", time.time() - start)

# NumPy array operation
a_np = np.arange(1_000_000)
b_np = np.arange(1_000_000)
start = time.time()
c_np = a_np + b_np
print("NumPy operation:", time.time() - start)
```

Result: NumPy is significantly faster (often 10-100x) for numerical operations!

1.7 반복 계산

1.7.1 반복계산이란?

Overview

This section explains various methods for performing iterative calculations in Python using `for` and `while` loops.

Key Concepts:

- **for loops:** Iterate over sequences or ranges
- **while loops:** Continue while condition is true
- **List comprehension:** Concise syntax for creating lists
- **Vectorization:** NumPy operations without explicit loops

Applications:

- Data processing
- Mathematical computations
- Algorithm implementation

1.7.2 for 문을 이용한 반복 계산

Basic for loop syntax:

```
# Sum from 1 to 10
total = 0
for i in range(1, 11):
    total += i
print("Sum from 1 to 10:", total)
# Output: 55

# List element squaring
numbers = [1, 2, 3, 4, 5]
squared = []
for num in numbers:
    squared.append(num ** 2)
print("Squared list:", squared)
# Output: [1, 4, 9, 16, 25]
```

Sum of Odd Numbers

```
print("Calculate sum of odd numbers from 0 to n.")
print("Enter n:")
n = int(input())

val = 0
for i in range(0, n + 1):
    if i % 2 == 1:  # Check if odd
        val = val + i

print("Sum of odd numbers up to n:", val)

# Example: n = 100
# Output: Sum of odd numbers up to n: 2500
```

Explanation:

- Loop through 0 to n
- Check if number is odd using modulo operator
- Add odd numbers to running total

1.7.3 while 문을 이용한 반복 계산

Basic while loop syntax:

```
while condition:
    # Code to execute repeatedly

# Example: Sum from 1 to n
print("Calculate sum from 1 to n.")
n = int(input("Enter n: "))
```

```
i = 1
total = 0
while i <= n:
    total += i
    i += 1

print(f"Sum from 1 to {n} is {total}.")

# Example: n = 100
# Output: Sum from 1 to 100 is 5050.
```

1.8 조건 계산

1.8.1 if else 구문

Overview

> Conditional calculation is a computation method that performs different operations or tasks based on specific conditions.

실습 코드: https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_08_conditionals.py

Basic if-else syntax:

```
if condition:
    # Code to execute if condition is True
else:
    # Code to execute if condition is False

# Example
num = int(input("Enter a number: "))
if num % 2 == 0:
    print(f"{num} is even.")
else:
    print(f"{num} is odd.")
```

1.8.2 다중 조건문 (if else else 구문)

Multiple Conditions with elif if-elif-else structure:

```
scores = [95, 82, 67, 58, 89]
grades = []

for score in scores:
    if score >= 90:
        grades.append('A')
    elif score >= 80:
        grades.append('B')
    elif score >= 70:
```

```
        grades.append('C')
    else:
        grades.append('D')

print("Scores:", scores)
print("Grades:", grades)
# Output: Scores: [95, 82, 67, 58, 89]
#         Grades: ['A', 'B', 'D', 'D', 'B']
```

1.8.3 if 구조 예문

```
n = 5
if n > 0:  # Conditional block starts
    print("n is positive")

for i in range(n):  # Loop block starts
    print(f"Iteration {i}")
    print("Loop finished")

print("Outside all blocks")
```

Output:

```
n is positive
Iteration 0
Iteration 1
...
Loop finished
Outside all blocks
```

1.8.4 비교연산자

Comparison Operators

Operator	Meaning	Example	Result
==	Equal	5 == 5	True
!=	Not equal	5 != 3	True
>	Greater than	5 > 3	True
>=	Greater or equal	5 >= 5	True
<	Less than	3 < 5	True
<=	Less or equal	3 <= 5	True

Usage:

- Conditional statements (if, elif, else)
- While loops
- Boolean expressions

1.9 입출력

1.9.1 키보드 입력 및 화면으로 출력

https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_09_io.py

Overview

This section explains methods for handling data input and output in Python.

Keyboard Input and Screen Output:

```
# Input from user
name = input("Enter your name: ")
age = int(input("Enter your age: "))

# Output to screen
print(f"Hello, {name}.")
print(f"You are {age} years old.")

# Example:
# Enter your name: Seejo Kim
# Enter your age: 22
# Output: Hello, Seejo Kim.
#         You are 22 years old.
```

1.9.2 파일로 입력 및 파일로 출력

File Input and Output Writing to a file:

```
# File writing
with open("output.txt", "w") as file:
    file.write("Hello.\n")
    file.write("Python file I/O example.\n")

# File reading
with open("output.txt", "r") as file:
    content = file.read()

print("File content:\n", content)
# Output:
# File content:
# Hello.
# Python file I/O example.
```

File modes: 'w' (write), 'r' (read), 'a' (append)

1.9.3 함수 I/O 사용하기

Matrix Input/Output Saving and loading matrices using NumPy:

```
import numpy as np

# Create matrix
A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Created matrix A:")
print(A)

# Save to file
np.savetxt("matrix.txt", A, fmt="%d", delimiter=",")
print("\nMatrix A saved to 'matrix.txt'.")

# Load from file
loaded_A = np.loadtxt("matrix.txt", delimiter=",", dtype=int)
print("\nMatrix A loaded from file:")
print(loaded_A)
```

1.10 함수(Function)

1.10.1 내장함수와 사용자 정의 함수

Overview

> Functions are independent blocks that perform specific tasks, used to modularize repetitive work and improve code reusability and readability.

https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_10_functions.py

Basic function syntax:

```
def function_name(parameters):
    # Function body
    return value

# Example
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

print("5! =", factorial(5))  # Output: 5! = 120
print("7! =", factorial(7))  # Output: 7! = 5040
```

Built-in Functions vs User-Defined Functions Built-in Functions:

- Python provides many built-in functions

- Examples: `len()`, `sum()`, `max()`, `min()`, `print()`
- Can be used directly without definition

User-Defined Functions:

```
# Define function
def f(x):
    return 9 - x * (x - 10)

def fprime(x):
    return -2 * x + 10

# Use functions
print("f(3) =", f(3))          # Output: f(3) = 0
print("f'(3) =", fprime(3))    # Output: f'(3) = 4
```

1.10.2 파이썬과 NumPy의 내장함수 종류

1.10.3 사용자 정의 함수 선언과 특징

1.10.4 사용자 정의 함수 예제

1.11 NumPy를 이용한 벡터 행렬

Overview

This section explains how to process vector and matrix operations using NumPy, Python's high-performance numerical computation library.

Key Topics:

- Dot product (inner product)
- Cross product (outer product)
- Distance between vectors
- Matrix multiplication

Advantages of NumPy:

- Vectorized operations (no explicit loops needed)
- High-dimensional array support
- Efficient linear algebra operations

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_11_numpy_linear_algebra.py

1.11.1 내적 (Dot Product)과 외적 (Cross Product)

Dot Product For n-dimensional vectors:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{n} a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n \tag{1.11.1-1}$$

```
import numpy as np

# Define two vectors
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Calculate dot product
dot_product = np.dot(a, b)
print("Dot product of a and b:", dot_product)
# Output: 32
```

Applications:

- Calculating angle between vectors
- Vector projections
- Similarity measures

Cross Product Cross product (3D vectors only):

```
# Define two 3D vectors
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Calculate cross product
cross_product = np.cross(a, b)
print("Cross product of a and b:", cross_product)
# Output: [-3  6 -3]
```

Properties:

- Result is perpendicular to both input vectors
- Magnitude represents area of parallelogram
- Only defined for 3D vectors
- Anti-commutative: $\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$

1.11.2 두 벡터의 거리 구하기

Euclidean distance for n-dimensional vectors:

$$\|\mathbf{a} - \mathbf{b}\| = \sqrt{\sum_{i=1}^{n} (a_i - b_i)^2} \tag{1.11.2-1}$$

```
import numpy as np

# Define two vectors
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Calculate Euclidean distance
distance = np.linalg.norm(a - b)

print("Vector a:", a)
print("Vector b:", b)
print("Distance between vectors:", distance)
# Output: 5.196152422706632
```

1.11.3 두 행렬의 곱(matrix multiplication)

For $m \times n$ matrix A and $n \times p$ matrix B:

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj} \tag{1.11.3-1}$$

```
import numpy as np

# Define matrices
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Matrix multiplication
C = A @ B  # or np.dot(A, B)
print("Matrix product:\n", C)
# Output:
# [[19 22]
#  [43 50]]
```

Note: Use `@` or `np.dot()` for matrix multiplication, not `*` (element-wise)!

1.12 객체지향 프로그래밍(OOP)

1.12.1 클래스 개요

Overview

> OOP is a programming paradigm that models data as objects and designs programs through interaction between objects.

Key Concepts:

Encapsulation Bundling data and methods together, implementing information hiding

Inheritance Child classes inherit characteristics from parent classes, increasing code reusability

Polymorphism Using the same method name to perform different operations for different objects

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_12_oop.py

Benefits:

- Code reusability
- Easier maintenance
- Better organization

1.12.2 클래스 선언

1.12.3 속성(변수)

1.12.4 클래스 메서드(함수)

Class methods operate at the class level:

```
class Example:
    class_variable = 0

    def __init__(self, value):
        self.instance_variable = value

    @classmethod
    def increment_class_variable(cls, amount):
        cls.class_variable += amount
        print(f"Class variable updated to {cls.class_variable}")

    @classmethod
    def create_instance(cls, value):
        return cls(value)

# Usage
Example.increment_class_variable(10)
# Output: Class variable updated to 10

instance = Example.create_instance(20)
print(instance.instance_variable)  # Output: 20
```

1.12.5 객체지향 프로그래밍(OOP) 예제 1

```
# Define class
class Animal:
    def __init__(self, name):  # Constructor
```

```
        self.name = name

    def speak(self):
        print(f"{self.name} is making a sound.")

# Inheritance
class Dog(Animal):
    def speak(self):  # Method overriding (polymorphism)
        print(f"{self.name} says Woof!")

# Create object and use
dog = Dog("Buddy")
dog.speak()
# Output: Buddy says Woof!
```

1.12.6 객체지향 프로그래밍(OOP) 예제 2

1.13 matplotlib를 이용한 그래프 시각화

Overview

This section explains how to express data as graphs using Matplotlib, Python's data visualization library.

Supported Graph Types:

- Line graphs
- Bar charts
- Histograms
- Scatter plots
- 3D surface plots

Key Components:

- Title, axis labels, legends
- Plot styles and colors
- Subplots for multiple visualizations

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter01/section_01_13_matplotlib.py

1.13.1 그래프 그리기 위한 주요 내장함수

1.13.2 2차원 그래프

2D Line Graph Example

```
import numpy as np
import matplotlib.pyplot as plt

# Generate data
x = np.linspace(-10, 10, 500)
y = x**2

# Create graph
plt.figure(figsize=(10, 6))
plt.plot(x, y, label='f(x) = x^2', color='blue', linewidth=2)

# Add annotations
plt.title('Line Graph of f(x) = x^2', fontsize=14)
plt.xlabel('x', fontsize=12)
plt.ylabel('f(x)', fontsize=12)
plt.axhline(0, color='black', linestyle='-', linewidth=0.8)
plt.axvline(0, color='black', linestyle='-', linewidth=0.8)
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend(fontsize=12)

plt.show()
```

1.13.3 2차원 산점도

2D Scatter Plot

```
import numpy as np
import matplotlib.pyplot as plt

# Generate data
np.random.seed(42)
x = np.random.rand(100) * 10
y = 2 * x + np.random.normal(0, 3, 100)

# Create scatter plot
plt.figure(figsize=(10, 6))
plt.scatter(x, y, color='blue', alpha=0.7, label='Data points')

# Add annotations
plt.title('2D Scatter Plot', fontsize=14)
plt.xlabel('x', fontsize=12)
plt.ylabel('y', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend(fontsize=12)

plt.show()
```

1.13.4 3차원 그래프

3D Surface Plot

```
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
import matplotlib.pyplot as plt

# Prepare data
x = np.linspace(-5, 5, 50)
y = np.linspace(-5, 5, 50)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

# Create 3D graph
fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
ax.plot_surface(X, Y, Z, cmap="viridis")

ax.set_title("3D surface")
ax.set_xlabel("X axis")
ax.set_ylabel("Y axis")
ax.set_zlabel("Z axis")

plt.show()
```

1.14 연습문제